

Project Submission Pro-Forma

Student name: Wei Luo

Student ID: 5712857

I wish the dissertation to be considered for the course (select one only):

- ☒ MSc in Applied Artificial Intelligence
- ☐ MSc in Cyber Security Engineering
- ☐ MSc in Cyber Security Management
- ☐ MSc in e-Business Management
- ☐ MSc in Engineering Business Management
- ☐ MSc in Games Engineering
- ☐ MSc in Healthcare Operational Management
- ☐ MSc in Innovation & Entrepreneurship
- ☐ MSc in International Trade, Strategy and Operations
- ☐ MSc in Management in Business Excellence
- ☐ MSc in Programme & Project Management
- ☐ MSc in Smart, Connected and Autonomous Vehicles
- ☐ MSc in Supply Chain & Logistics Management
- ☐ MSc in Sustainable Automotive Engineering

I confirm that I have included in my dissertation:

- ☐ An abstract of the work completed
- ☐ A declaration of my contribution to the work and its suitability for the degree
- ☐ A table of contents
- ☐ A list of figures & tables (if applicable)
- ☐ A glossary of terms (where appropriate)
- ☐ A clear statement of my project objectives
- ☐ A full reference list (the [Harvard referencing style is recommended for WMG](#))
- ☐ An appendix containing email confirmation of ethical approval or waiver

If receiving ethical approval, the ethical approval number for this research is: **insert reference number by replacing this highlighted text**. If ethical approval was waived, then please write 'Ethical Approval Waived'. Remember to include your email confirmation of ethical approval or waiver in the **Appendix of your submission**.

I understand that this dissertation will be stored for potential access by third parties e.g. staff/student training purposes, access by external examiners and professional statutory and regulatory bodies.

Signed:

Date:

Commented [A1]: This dissertation template is for **full-time MSc students enrolled on a WMG course for the 25-26 academic year**. Please read this comment with care.

Prior to submitting your dissertation on Tabula, replace the text highlighted in yellow in this template as well as delete the instructional comments.

Support for the Dissertation

- For questions about the dissertation, you can contact your supervisor or Course Leader.

- For guidance on writing each chapter of the dissertation, view the 'Postgraduate dissertation checklist' here: <https://moodle.warwick.ac.uk/mod/book/view.php?id=2662557>

- For support on the dissertation marking criteria, view the checklists available at the link above. You can also view the dissertation marking criteria here: <https://warwick.ac.uk/fac/sci/wmg/ftmsc/project/exam>



A Controlled Evaluation of Large Language Models for Full-Stack Software Development: Architectural Consistency Analysis

by

Name Surname, qualifications obtained

Dissertation submitted in partial fulfilment for the Degree of Master of Science in Applied Artificial Intelligence

name/department of
collaborating partner if applicable

WMG
University of Warwick

Submitted month year

Declaration

I have read and understood the rules on cheating, plagiarism and appropriate referencing as outlined in my handbook and I declare that the work contained in this assignment is my own, unless otherwise acknowledged.

No substantial part of the work submitted here has also been submitted by me in other assessments for this or previous degree courses, and I acknowledge that if this has been done an appropriate reduction in the mark I might otherwise have received will be made.

Project definition for my degree (as copied from the Specific Course Regulations for your course, available here: <https://warwick.ac.uk/fac/sci/wmg/ftmsc/opregs/courseregs/>)

The MSc Applied Artificial Intelligence project must involve an investigation of a relevant AI topic that demonstrates technical depth, meets academic standards, and addresses a well-defined research question in one or more core AI areas—such as machine learning, deep learning, natural language processing, computer vision, generative AI, or AI for industry and robotics. It must also show critical awareness of the ethical, societal, or deployment implications of AI technologies. This can be achieved through the design, implementation, and evaluation of a technical AI solution or through a systematic and critical literature review that synthesises current research and practice. Regardless of the approach, students must apply appropriate research methods, engage critically with existing literature, demonstrate depth of understanding, and justify the relevance and rigour of their work in line with academic standards.

My project relates to this definition in the following way:

Replace this text highlighted in yellow with your justification for how your independent research project relates to the project requirements for your course. You must include key words from the project requirements in your answer.

This declaration must be limited to one page.

Abstract

Insert your abstract here by replacing the highlighted text. You can find information on how to write an abstract in Chapter 10 of *Succeeding with Your Master's Dissertation* by John Biggam (2021), [available online via the University of Warwick library](#).

An abstract is typically limited to one page.

Acknowledgements

While it is not a dissertation requirement to include acknowledgements, expressing your appreciation for support provided by a supervisor, an organisation, family, friends, and/or other individuals and groups that may have positively influenced your research experience and the writing of the dissertation is an important professional behaviour. You can replace this highlighted text with your own acknowledgements.

The acknowledgements are typically limited to one page.

Table of Contents

Project Submission Pro-Forma..... 1

Declaration.....iii

Abstract.....iv

Acknowledgements.....v

Table of Contents.....vi

List of tables.....viii

List of figures.....ix

Chapter 1: Introduction 1

1.1 Background 1

1.2 Problem Statement 1

1.3 Research Gap 1

1.4 Research Question and Objectives 2

1.5 Contributions and Expected Outcomes 2

1.6 Structure of dissertation 2

Chapter 2: Literature Review 4

2.1 Architectural Erosion and Its Measurement Under AI-Driven Development..... 4

2.1.1 Architectural Erosion..... 4

2.1.2 Measuring Architectural Conformance..... 4

2.2 Evaluating LLM-Based Code Generation: From Function Completion to Autonomous Agents..... 5

2.3 Synthesis and Research Gap 6

Chapter 3: Methodology..... 6

3.1 Research Design 7

3.2 Starter Codebase..... 7

3.3 Sprint Chain Design 8

3.4 Rulepack..... 9

3.4.1 Measurement Layers..... 9

3.4.2 Architectural Concerns..... 9

3.4.3 Rule Nomenclature and Implementation..... 10

3.5 Experimental Setup 12

3.5.1 Controlled Variables..... 12

3.5.2 Prompt Design..... 12

3.5.3 Execution Protocol 13

3.6 Threats to Validity 13

3.7 Ethics and Summary..... 14

3.7.1 Ethical Considerations..... 14

3.7.2 Summary 14

Chapter 4: Results and Analysis 14

4.1 Experimental Runs and Data Quality 14

4.2 Descriptive Overview of Architectural Violations 16

4.3 Effect of Prompt Strategy..... 17

4.4 Differences Between Coding Agents..... 18

4.5 Longitudinal Trajectories Across T1 to T3 18

4.6 Silent Decay: Constraint and Metric Cross-Classification..... 19

4.7 Secondary Mechanism Analyses 20

4.8 Functional and Efficiency Outcomes 21

4.9 Summary of Findings..... 21

Chapter 5: Discussion..... 23

5.1 Introduction 23

5.2	Interpreting the Principal Findings.....	23
5.2.1	Architectural erosion under agents is immediate and mechanism-specific	23
5.2.2	Prompt-embedded rules did not secure conformance.....	23
5.2.3	Agents exhibit distinguishable architectural profiles	23
5.2.4	Degradation follows task structure, with partial self-correction	24
5.2.5	The empty silent-decay class is a boundary condition, not a refutation	24
5.2.6	Completion claims are not verified outcomes	24
5.3	Limitations.....	25
5.4	Future Work.....	25
5.5	Summary	26
Chapter 6:	Conclusion.....	27
6.1	Summary of the Study.....	27
6.2	Answers to the Research Question and Objectives	27
6.3	Contributions	28
6.4	Closing Remarks	28
References		29
Appendices.....		31
Appendix A:	Insert a title for your Appendix A by replacing this highlighted text in yellow.....	31

List of tables

Table 3.1 Summary of Measurement Layers 7

Table 3.2 Summary of Specific Tasks and Targeted Architectural Concerns Across Sprints 9

Table 3.3 The Concerns × Layers Matrix of the Rulepack 10

Table 3.4 Prompt Block Composition..... 12

Table 5.1 Replace this text highlighted in yellow with the title of table **Error! Bookmark not defined.**

List of figures

Figure 3.1 Experimental Workflow of the Study 13

Figure 2.1 Replace this text highlighted in yellow with the title of figure (source, if needed) ... Error! Bookmark not defined.

Chapter 1: Introduction

1.1 Background

The landscape of software development has witnessed a fundamental transformation with the emergence of Large Language Model (LLM)-powered coding tools. Tools such as GitHub Copilot, Cursor, Claude Code, and OpenAI Codex have moved beyond experimental prototypes into daily industrial use, with millions of developers now relying on AI-generated code suggestions, automated refactoring, and autonomous task completion as part of their standard workflow (Jalil, 2025). One study reports a 24% productivity gain among student developers using GitHub Copilot in brownfield tasks (Shihab et al., 2025), and organisational adoption continues to accelerate across both startups and established enterprises.

The scope of these tools has expanded in parallel with their deployment. Early code-completion assistants operated at the level of individual functions, but research has since extended their scope to the repository level, in which models reason over multi-file dependencies and project-level architecture to produce code that fits within a larger codebase (Liao et al., 2024; Wang et al., 2025). More recently, a significant shift has occurred from code-completion assistants towards fully autonomous coding agents (Li, Zhang and Hassan, 2025). Unlike earlier tools that suggest individual lines or functions in response to developer prompts, modern coding agents such as Claude Code and Codex operate autonomously: given a task description, they read the existing codebase, plan a sequence of code changes, edit multiple files, run tests, observe failures, and iteratively refine their output until the task is complete, without human intervention during execution. The shift to autonomous execution represents a qualitative change in the developer's role, from writing code to directing and reviewing AI-generated systems.

However, the rapid adoption of these tools has outpaced the research community's ability to evaluate a specific dimension of software quality: whether AI-generated code preserves architectural consistency, keeps components properly coordinated, and remains maintainable over the long term. These are the qualities that distinguish production-grade software from disposable prototypes.

1.2 Problem Statement

Recent evaluation of LLM coding capabilities is dominated by benchmark-oriented assessment against unit tests (Jiang et al., 2026). Standardised benchmarks evaluate the functional correctness of generated code. Their scope has expanded progressively from single functions to repository-level patches, and now to agent workflows over live codebases (Xia et al., 2024; Le et al., 2026). However, evaluation still relies on test-based verification (Deng et al., 2025; Le et al., 2026).

In practice, passing a test suite verifies only functional correctness, not architectural consistency. A codebase may pass its tests while still violating structural rules, breaking API contracts, or eroding modularity and long-term maintainability (de Silva and Balasubramaniam, 2012; Esposito et al., 2026). Such structural properties have long been recognised in the software engineering community as both objectively measurable and essential to long-term system quality (Li et al., 2014), but they remain largely unassessed in current LLM code assessment (Jiang et al., 2026). The unassessed structural dimension widens further under the agent-based regime, where a single task may edit multiple files within one conversational context, and successive iterations accumulate on the evolving codebase state (Xia et al., 2024; Le et al., 2026). Current benchmarks, however, provide no mechanism to observe such erosion as it unfolds (Recupito et al., 2024; Li, Zhang and Hassan, 2025).

1.3 Research Gap

These unassessed structural properties directly affect how full-stack development can be safely delegated to autonomous coding agents. If coding agents reliably preserve architectural integrity across iterative development, organisations can confidently delegate larger portions of full-stack development to autonomous systems. If they do not, or if their behaviour depends materially on prompt structure or task, then developers require evidence-

based guidance on where and how these tools can be deployed without accumulating structural debt at a rate that outpaces human review.

Currently, systematic evidence remains insufficient for the class of agent-based, iterative full-stack development that increasingly characterises industrial practice. Therefore, there is a need for a controlled, longitudinal, and architecture-focused investigation into how coding agents behave when tasked with the continuous evolution of a full-stack codebase, as well as the conditions that shape their architectural behaviour.

1.4 Research Question and Objectives

The research question guiding this study is as follows:

How effectively can LLM-based coding agents maintain architectural consistency and API-contract integrity across a sequence of iterative, cumulative full-stack development tasks?

The study is guided by five research objectives:

RO1 · To construct a controlled longitudinal experimental framework for measuring architectural drift under agent-driven full-stack development.

RO2 · To develop a multi-dimensional conformance measurement instrument calibrated for JavaScript/TypeScript full-stack architecture. The instrument organises architectural concerns across backend, frontend, and cross-stack domains.

RO3 · To characterise the longitudinal trajectory of architectural conformance under two state-of-the-art coding agents.

RO4 · To evaluate the effect of prompt structure on the observed conformance trajectory.

RO5 · To derive evidence-based practitioner guidance for the deployment of coding agents on iterative full-stack development.

1.5 Contributions and Expected Outcomes

The evaluation methodology introduced here assesses the long-horizon planning consistency and structural discipline of autonomous coding agents during cumulative software evolution. Rather than providing a universal benchmark dataset, we propose a reusable assessment paradigm and demonstrate its feasibility using a custom-developed, representative full-stack business system as an experimental exemplar.

For engineering teams considering the delegation of development tasks to autonomous agents, this study offers a methodological blueprint to transition from purely functional testing to structural code quality audits.

Practitioners can readily adapt the proposed architectural assertion strategy and reference rulepack to design custom, domain-specific evaluation oracles for auditing agents on their own proprietary codebases.

1.6 Structure of dissertation

This dissertation is organised as follows:

Chapter 2 — Literature Review traces the evolution of architectural erosion as a measurable phenomenon and its acceleration under AI-driven development, surveys the mature but stack-mismatched tradition of architectural conformance measurement, and reviews contemporary LLM code-generation benchmarks to identify three specific evaluation dimensions they leave open. The chapter identifies the research gap this study occupies and previews the three-component response developed in Chapter 3.

Chapter 3 — Methodology develops the study as a controlled longitudinal experiment. It presents the overall research design and its three coupled components: a bespoke JavaScript/TypeScript full-stack starter codebase, a five-sprint task chain that drives open-ended evolution and a multi-dimensional rulepack across backend, frontend, and cross-stack concerns.

Chapter 4 — Results and Analysis organised by evaluation dimension, culminating in the cross-level capability decay curves that constitute the study's primary empirical output.

Chapter 5 — Limitations and Future Work

Chapter 6— Conclusion

Chapter 2: Literature Review

2.1 Architectural Erosion and Its Measurement Under AI-Driven Development

2.1.1 Architectural Erosion

Software architecture is not an artefact produced once at project outset but a set of constraints that every subsequent modification must observe. Perry and Wolf (1992) define architecture as a triple of elements, form, and rationale, where the form component constrains the choice and placement of architectural elements. For instance, the rule that a controller may invoke a service but not access the data layer directly constitutes an architectural constraint that applies to every developer across every iteration of the system.

In practice, mainstream tooling is not designed to enforce this class of constraint. Compilers verify type correctness, test suites verify functional behaviour, and linters verify surface-level style, but none of them detects whether a new feature has introduced an undeclared cross-module dependency or placed business logic in the wrong layer. As de Silva and Balasubramaniam (2012) observe, mainstream programming languages provide no dedicated mechanism for enforcing project-specific architectural communication rules during code modification: general-purpose encapsulation features such as access modifiers can restrict low-level visibility, but they cannot express or enforce a constraint such as 'a controller may invoke a service but not access the data layer directly'. Consequently, architectural violations accumulate in the codebase without triggering any automated check. Prior work indicates that such accumulation correlates with tighter coupling, higher defect counts, and reduced modularity (Eick *et al.*, 2001; de Silva and Balasubramaniam, 2012).

The introduction of autonomous coding agents does not displace this dynamic but accelerates it. Even before AI entered the toolchain, Wan *et al.* (2023) observed that under time pressure, even humans equipped with implicit architectural understanding and subject to peer review routinely trade structural integrity for faster feature delivery.

Coding agents amplify each factor in this trade-off along three dimensions. First, they modify code at rates that substantially exceed human review capacity (Li *et al.*, 2025). Second, each iteration typically begins without a persistent, structured record of the architectural rationale behind earlier changes, relying instead on whatever can be inferred by re-reading the current code state (Zeng *et al.*, 2025; Le *et al.*, 2026). Third, their behaviour is shaped predominantly by execution-based feedback such as test-pass signals, error resolution, and task completion, each of which rewards precisely the shortcut-taking that Wan *et al.* (2023) identified as the mechanism of erosion.

Recent evaluations of agentic systems have begun to flag the resulting risks. Esposito *et al.* (2026) warn of architectural degradation arising from overuse of or uncritical reliance on AI-generated recommendations, while Li *et al.* (2025) identify the long-term impact of AI-authored code on software quality as an open empirical question. The operational scope for such degradation is already established: Xia *et al.* (2024) demonstrate that contemporary agents routinely handle complex, multi-location repository-level tasks, making architectural drift a concrete rather than hypothetical concern. In contrast, practitioner awareness still lags behind: Recupito *et al.* (2024) report that AI-introduced technical debt is perceived as low in prevalence yet high in severity, indicating that the phenomenon outpaces the community's capacity to detect it. Thus, traditional oversight mechanisms were not initially designed to detect architectural violations mechanically. Built around human-scale editing rhythms and human-mediated code review, these mechanisms are now further challenged by agents whose speed and limited architectural memory turn each iteration into a fresh opportunity for structural drift.

2.1.2 Measuring Architectural Conformance

Measuring architectural conformance is a mature and empirically validated area of software engineering. The typical paradigm extracts structural information statically from source-code dependency graphs and compares it against a description of the intended architecture (Knodel and Popescu, 2007). These approaches include reflexion models, which compare a planned architectural model against a source-code model derived from the

actual implementation; relation conformance rules, which specify permitted and forbidden relations between components; and component access rules, which restrict which components may invoke which ports.

Building on this foundation, more recent work embeds conformance checking directly into the development pipeline. Bucaioni et al. (2024) formalise the notion of continuous conformance as a distance function quantifying how closely an implemented architecture adheres to a designated reference architecture and describe approaches that trigger the conformance process at each code integration. Alongside this constraint-based tradition, Li et al. (2014) demonstrate empirically that modularity metrics computed directly from source code correlate significantly with an independent ground-truth measure of architectural technical debt. Constraint checking and metric-based measurement thus together define the state of the art.

A clear technology-stack limitation nonetheless persists: the tradition has matured predominantly around the object-oriented, class-and-package abstractions of Java and C/C++. Li et al. (2022), for instance, report in their systematic mapping study of architecture-erosion research that more than half of the tools proposed for detecting architecture erosion support only a single programming language, reflecting a broader concentration of conformance tooling on individual, typically statically typed, class-based ecosystems. The JavaScript and TypeScript stack, meanwhile, now underpins an increasing share of contemporary full-stack development: TypeScript adoption alone has risen from 12% in 2017 to 35% in 2024 (JetBrains, 2024), with the rise of coding agents further accelerating full-stack development on the JS/TS stack. The growing dominance of the JS/TS stack exposes a gap. The abstractions underlying prior approaches, such as packages, class hierarchies, statically resolvable imports, map only partially onto the module systems, dynamic imports, and framework-mediated composition typical of the JS/TS ecosystem, and findings validated on other programming languages cannot be transferred wholesale to this setting.

2.2 Evaluating LLM-Based Code Generation: From Function Completion to Autonomous Agents

To evaluate the code-generation capabilities of LLM-based systems, the research community has proposed several benchmarks. Unlike earlier benchmarks focused on function-level code generation, such as HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021), benchmarks for repository- and agent-level tasks place a greater emphasis on simulating realistic development scenarios that involve cross-file dependencies, iterative refinement, and long-horizon task completion. SWE-Bench (Jimenez et al., 2024) constructs 2,294 real-world software engineering issues drawn from twelve Python repositories, tasking agents with producing patches that are evaluated via the original unit tests. SWE-Bench Pro (Deng et al., 2025) extends this direction with industrial-grade tasks of substantially longer horizon, again arbitrated by execution against a curated test suite. SWE-EVO (Le et al., 2026) further advances the frontier by framing tasks as the release-note delta between one tagged version and its next, evaluating agents on their ability to reproduce the resulting evolved repository state through a comprehensive suite of FAIL→PASS and PASS→PASS tests. In a related direction, Commit0 (Zhao et al., 2025) presents agents with a code skeleton and a requirements document drawn from real GitHub projects, tasking them with completing the code to pass the original tests. Together, these benchmarks have driven the field from single-function completion to trajectory-level agent behaviour over live codebases.

Read as a foundation for studying architectural preservation under agent-driven iterative development, though, these benchmarks exhibit three notable limitations:

1. Task Isolation. Issue-resolution benchmarks such as SWE-Bench (Jimenez et al., 2024) and SWE-Bench Pro (Deng et al., 2025) score each task in isolation against a fixed repository snapshot by the number of tests passed. Little state is preserved across tasks, and no mechanism observes whether an agent's successive contributions gradually accumulate structural cost over a development sequence. Therefore, current evaluation captures momentary correctness rather than the trajectory of a system under continuous AI-assisted change.

2. Evaluation Reduction to Functional Signals. The dominant criterion of

remains execution against a test suite. Jiang et al. (2026) observe that current benchmarks primarily assess functional correctness and provide no comprehensive coverage of other quality dimensions, while Deng et al. (2025) themselves acknowledge, in the limitations section of SWE-Bench Pro, the need for evaluation approaches that extend beyond test-based verification to capture adherence to software engineering best practices. A codebase whose tests pass may nonetheless have violated the architectural constraints it began with, so test-based evaluation does not measure the structural dimension of software quality at all, because this dimension falls outside the scope of the evaluation framework.

3. Replay-Based Evolution. SWE-EVO (Le et al., 2026) tasks the agent with reproducing a known release delta and evaluates success against the test suite associated with that delta. The replay design, though, presupposes a fixed reference endpoint against which every trajectory is scored, and therefore cannot capture the open-ended condition under which architectural decisions genuinely accumulate. Li, Zhang and Hassan (2025) frame the underlying gap more directly, identifying the long-term impact of AI-generated code on software quality as an open empirical question.

Taken together, these three limitations indicate that current benchmarks, though driving rapid progress on functional generation, are not yet positioned to answer questions about how an agent's decisions accumulate structurally over iterative development.

2.3 Synthesis and Research Gap

In light of these limitations, our research is dedicated to constructing an evaluation framework for a dimension that current benchmarks and current conformance tooling both leave open: the longitudinal trajectory of architectural conformance under agent-driven, open-ended full-stack development. We instantiate this apparatus as three coupled components, each responding to a specific shortcoming identified above.

First, we develop a bespoke JavaScript/TypeScript starter codebase, verified violation-free against the study's own conformance instrument prior to execution (3.2). The starter codebase targets the stack-coverage gap in existing conformance tooling and ensures that any structural drift observed downstream can be attributed to the agent under test rather than inherited from legacy debt.

Second, we design a five-sprint task chain executed without workspace reset between iterations (3.3). Each sprint's terminal workspace becomes the exact input to the next, and no reference release version is supplied against which the agent's trajectory is scored. The no-reset sprint chain replaces the task-isolated and replay-based framings of existing benchmarks with an open-ended evolution condition under which architectural decisions genuinely accumulate.

Third, we construct a multi-dimensional rulepack, organised across backend, frontend, and cross-stack concerns and along two measurement layers, namely binary constraint violations and continuous drift metrics (3.4). Compared with the constraint-checking and metric-based traditions surveyed in 2.1.2, the rulepack extends this measurement approach specifically to cross-stack architectural properties, making visible the structural quality dimension that test-based arbitration cannot see.

Together, these three components form an evaluation apparatus calibrated to the language regime, task horizon, and structural dimension that current traditions leave open. Chapter 3 develops each component in detail and specifies the experimental conditions under which the apparatus is exercised.

Chapter 3: Methodology

To characterise how LLM coding agents preserve or erode architectural consistency across long-horizon software evolution, this chapter presents a controlled longitudinal experiment.

3.1 Research Design

As established in 2.2–2.3, existing evaluation paradigms — issue-resolution benchmarks such as SWE-Bench (Jimenez et al., 2024) and SWE-Bench Pro (Deng et al., 2025), and evolutionary paradigms such as SWE-EVO (Le et al., 2026) — leave task isolation, evaluation reduction to functional signals, and replay-based framing unresolved for the study of long-horizon architectural behaviour. Beyond these methodological gaps, as described in 2.1.2, industrial architecture-compliance tooling has matured on enterprise Java and C++ codebases; on the JavaScript/TypeScript stack, comparable framework-level support remains limited. Nor can manual review, which catches architectural drift in human-authored development, scale to the rate at which agents produce code. To address these conditions, this study designs three components: a bespoke starter codebase, a three-sprint task chain, and a multi-dimensional rulepack.

- 1. The starter codebase (attributable starting state). It is a bespoke, minimal customer relationship management (CRM) system implemented by the author as a two-tier full-stack application, comprising a NestJS/TypeScript backend and a React/JavaScript frontend. Prior to the experiment, the codebase is verified under the rulepack as containing zero architectural violations. The zero-violation baseline enables any downstream deviation observed during the sprint chain to be attributed to the agent under test rather than to inherited debt.
- 2. The sprint chain (controlled evolution trajectory). It consists of three sprints, denoted T1 through T3, executed sequentially against the same workspace. Between sprints, the workspace is not reset, cleaned, or curated: the codebase state produced by the agent at the end of sprint T_k becomes the exact input to sprint T_{k+1}. The no-reset protocol specifies the evolution trajectory rather than replaying historical commits. Each sprint embeds a specific development challenge chosen to expose a distinct class of architectural concern, enabling any observed deviation to be traced back to the sprint in which it emerged.
- 3. The rulepack (multi-dimensional conformance measurement). It organises 19 architectural concerns across two measurement layers. Constraint-layer rules encode binary, mechanically decidable violations (e.g., controllers importing repositories directly). Metric-layer rules capture continuous drift signals expressed as ratios or densities (e.g., deep-import count divided by total imports). Together, the two layers surface architectural properties that no test suite is designed to observe.

Table 3.1 Summary of Measurement Layers

Layer	What it measures
Constraints	Binary, mechanically decidable violations
Metrics	Continuous drift signals (ratios/densities)

3.2 Starter Codebase

The starter codebase serves as the fixed origin of the sprint chain: what stack it runs on and what domain it models.

1. Architectural paradigm and business domain

Software architecture in industrial practice is not monolithic: recent surveys catalogue over 120 distinct reference architectures developed by academia and industry (Bucaioni et al., 2022), spanning layered, microservice, event-driven, and serverless styles among others (Wan et al., 2023). Each style imposes a different set of constraints and thus a different notion of what "architectural conformance" means. This study focuses on a specific architectural paradigm: a decoupled client-server application comprising a layered backend and a frontend client that interact via a REST interface. This paradigm is adopted because it remains the dominant architecture for full-stack SaaS applications in production use.

A simulated CRM platform is applied as the reference system because its evolving entity-relationship model inherently demands a decoupled, layered architectural design. The entity model (Company and Contact at

initialisation, with Deal and their interlinking added across the task chain) requires the architecture to keep data properly encapsulated across layers, enforce transaction boundaries at the service level, and keep client- and server-side Data Transfer Objects (DTOs) consistent across successive development iterations.

2. Technology stack

Within the JavaScript/TypeScript ecosystem, the codebase adopts React on the frontend and NestJS on the backend. React is the most widely used web framework worldwide (Stack Overflow, 2024), and its dominant position makes it a defensible representative of the frontend agents most often encounter. NestJS has become an increasingly adopted backend framework in the Node.js ecosystem in recent years (Devographics, 2025). Its deep integration with TypeScript and its module-based composition offer explicit architectural discipline that addresses the fragmented architecture and lack of structural standardisation in traditional Node.js projects. The two tiers deliberately use different language regimes. The backend uses TypeScript, while the frontend uses plain JavaScript. The resulting language asymmetry allows the study to observe whether cross-stack contracts survive iterative development.

3. Rationale for a Bespoke Codebase

A major challenge for accurately measuring structural drift is the confounding variable introduced by existing open-source repositories. Specifically, mature codebases inevitably carry historical technical debt and architectural inconsistencies. Using such repositories makes it difficult to perform precise attribution; an observed layer violation might be attributed not to the agent's errant decision, but rather to the imitation of pre-existing flawed patterns. To address this limitation, the starter codebase is constructed from scratch. Before the experiment, it is verified against the rulepack detailed in 3.4 and recorded as a zero-violation baseline. Initialising the testbed in this verified state isolates the agent's contribution, so that any structural deviation observed during the cumulative sprints can be attributed to the agent under test rather than to inherited flaws.

3.3 Sprint Chain Design

To systematically investigate the accumulation of architectural erosion, the design of the sprint chain is governed by three core principles.

First, the three tasks through phases characteristic of evolving business systems: initial entity introduction, schema restructuring, and business-rule constraining. Progressing through these phases in sequence is essential to preserve the temporal dynamics of authentic software development workflows, thereby capturing architectural erosion as a true longitudinal phenomenon.

Second, to simulate realistic and demanding development scenarios, the specific challenges embedded within each sprint are deliberately curated from empirically documented triggers of architectural erosion. These include new entity introduction, schema evolution, cross-cutting business rules, and cross-stack contract changes (Wan et al., 2023; de Silva and Balasubramaniam, 2012).

Third, to meet the complex evaluation demands of end-to-end tasks, the three tasks collectively exercise the architectural constraints encoded in the predefined rulepack (3.4). The rulepack evaluates all 19 concerns on every snapshot regardless of task; each task's design instead targets a specific subset it is expected to stress most directly (Table 3.2). Exercising the rulepack's full constraint set in this way enables a fine-grained evaluation of structural compliance, allowing observed deviations to be precisely diagnosed at the level of specific architectural concerns rather than being obscured by a code quality score.

Furthermore, to ensure the assessment accurately measures an agent's true capabilities in a realistic setting, all tasks are specified as product-manager tickets. These specifications detail the required deliverables exclusively in high-level business terms, deliberately withholding any internal architectural or structural guidance. The specific tasks and their targeted architectural concerns are detailed in Table 3.2, followed by the comprehensive sprint-by-sprint specifications.

Each concern is identified by a short code of the form <STACK>-<CONCERN> (BE for backend, FE for frontend, CROSS for concerns spanning both). Table 3.2 summarises which of the rulepack's nineteen concerns each task is designed to stress; their full definitions and grounding are given in 3.4.2 and Table 3.3.

Table 3.2 Summary of Specific Tasks and Targeted Architectural Concerns Across Sprints

ID	Task Summary	Target Concerns	Design Rationale
Task1 (T1)	Introduce Deal entity with CRUD;	BE-STRUCT, BE-DEP, BE-CONTRACT, BE-ROUTE, CROSS-EP, CROSS-TYPE	Greenfield module isolates default architectural habits
Task2 (T2)	Convert Contact-Company and Deal-Contact relationships from 1:N to M:N; migrate existing data	BE-CONTRACT, BE-DOM, BE-DUP, CROSS-TYPE, CROSS-PROP	Bidirectional dependency on a shared resource
Task3 (T3)	Upgrade Deal.stage to a six-value state machine with unidirectional transitions and reason-annotated rollback	BE-DOM, BE-ERR, BE-DUP, CROSS-PROP	Shape change to existing field

After the T1 to T3 trajectory completes, the agent is asked to review the resulting workspace and report architecture-consistency findings. Task 4(T4) is therefore categorically different from T1–T3: it produces no code and is not scored by the rulepack. Its findings serve a single purpose: to test the agent's capacity to detect its own architectural problems, and how closely what it detects overlaps with the harness's independent measurement.

3.4 Rulepack

The evaluation instrument used in this study is a rulepack organised as a matrix of architectural concerns against measurement layers.

3.4.1 Measurement Layers

The primary axis of the matrix categorises the measurement, establishing how architectural drift is quantified. To capture both overt violations and subtle structural degradation, the framework decomposes into two distinct analytical roles.

Constraints (C): This layer encodes binary, mechanically decidable violations. These are definitive transgressions that can be enforced by static-analysis tooling without interpretation (e.g., controllers importing repositories directly, DTO fields lacking validation decorators, or frontend URL calls missing a matching backend endpoint). The primary longitudinal signal at this layer is the per-sprint violation count.

Metrics (M): Continuous drift signals are captured at this layer (e.g., deep-import count divided by total imports, prop-drilling depth on the frontend, or the symmetric difference between frontend and backend endpoint sets). While these quantities do not necessarily trigger binary violations, their trajectories reveal gradual erosion, which is the limit of traditional constraint checkers.

3.4.2 Architectural Concerns

The nineteen concerns are a deliberate selection, not an exhaustive one, since no fixed rule set can capture every structural property of an architecture, and selection is unavoidably a matter of judgement. The criterion applied here is susceptibility to silent erosion: a concern was included if its violation is something an iterative modification

could introduce without triggering a test failure or any other visible signal. Three explicit criteria operationalise this judgement:

- 1. Architectural, not stylistic. Violating it changes the system's dependency structure, module boundaries, or cross-component contracts, not only its formatting.
- 2. Mechanically decidable. It can be evaluated from the code alone, so it can be operationalised as both a binary constraint and a continuous metric without human interpretation.
- 3. Anchored in an existing basis, not an ad hoc judgement of importance. Most concerns draw on an established measurement construct from the software-engineering literature (Li *et al.*, 2014; Wan *et al.*, 2023): dependency direction and cyclic structure, information hiding at module boundaries, contract-based interfaces, code duplication, unit and nesting complexity, co-change propagation, and web-API evolution. Where a concern is instead framework-idiomatic, such as NestJS module composition or React state placement, no academic metric exists, but a developer-agreed best practice usually does. In these cases, the concern is anchored not in a citation but in that practice, as documented in the reference system's own conventions.

Several retained concerns also match failure modes that empirical studies report in AI-generated code: duplication (Mo *et al.*, 2025), elevated complexity (Li *et al.*, 2024), and recurring code smells (Siddiq *et al.*, 2024).

The partition into backend, frontend, and cross-stack domains follows the deployment structure of the system itself.

Backend Concerns (nine dimensions): data-contract stability, dependency direction, domain boundaries, resource/policy duplication, transaction and error handling, routing conventions, unit complexity, structural integrity, and test-construction discipline.

Frontend Concerns (seven dimensions): inter-component communication, component granularity, data-fetching and effects, component/logic reusability, routing structure, state location, and style isolation.

Cross-Stack Concerns (three dimensions): endpoint existence, request/type contract consistency, and change propagation completeness.

3.4.3 Rule Nomenclature and Implementation

Every rule in the rulepack is identified by a four-part name: <STACK>-<CONCERN>-<LAYER>-<ID>. STACK is one of FE, BE, or CROSS, naming the frontend, backend, or cross-stack domain; CONCERN (e.g. DOM for domain boundaries) and LAYER (C for Constraint, M for Metric) are drawn from measurement layers 3.4.1 already defines; ID is a sequential number (001, 002, ...) disambiguating multiple rules within the same concern and layer, and carries no meaning of its own. Table 3.3 summarises the full C-layer and M-layer matrix; the exhaustive text of all fifty-six rules is given in Appendix A.

Table 3.3 The Concerns × Layers Matrix of the Rulepack

Concerns	Layers	
	Constraints	Metrics
BE-CONTRACT: Data-contract stability	Every entity/relationship schema change requires a corresponding migration; DTOs use `class-validator` decorators; optional properties validate supplied values; input whitelisting is preserved	Ratio of DTO fields carrying a `class-validator` decorator to total DTO fields
BE-DEP: Layer Dependency Direction	Controller cannot import Repository directly; dependencies follow Controller to Service to Repository to Entity; no circular imports	Dependency-violation density = (direction-violating import edges + cyclic-dependency edges) / total import edges.

BE-DOM: Domain Boundaries	Inter-module communication occurs only via a module's public entry point (`.module.ts`/`index.ts`); a module's public entry point must not export repositories or entities	Count of deep-path imports that bypass a module's public entry point
BE-DUP: Code duplication density	Each business resource has exactly one backend owner (no competing modules, controllers, routes, or entity-table owners); each business policy or invariant has exactly one authoritative implementation; no copied equivalent production functions or code blocks	Clone ratio= duplicated-line coverage / total token-bearing production lines
BE-ERR: Transaction and Error Boundaries	Service layer must not throw `HttpException`; service failures use the project's unified `AppException`; no silent or log-only catch	Weighted exception-rule violation density = $\sum(\text{weight}_{(i)} \times \text{violation-count}_{(i)})$
BE-ROUTE: Interface Routing	Resolved public routes use the global `/api` prefix and kebab-case resource paths	Cyclomatic complexity ratio
BE-SIZE: Unit Complexity	Production methods have at most three direct parameters	Count of endpoints not matching the prefix convention
BE-STRUCT: Layered Structure	Controller–Service–Repository organisation is instantiated; the module registers all three in its `@Module` metadata	Ratio of modules missing a controller, service, or repository layer file to total modules
BE-TEST: Test Construction Discipline	Services obtain repositories through dependency injection; tests must not call `new Repository(...)` directly	Mock-per-test-case ratio = mock-usage count / test-case count.
FE-COM: Component granularity	Component files ≤ 300 non-blank lines; each React component has render-decision nesting depth ≤ 3 and emits at most one depth finding; pure structural JSX nesting is excluded	Average of each production React component's maximum render-decision nesting depth
FE-COMM: Inter-component Communication	No global event bus	Prop fan-out average = mean prop count of JSX elements at or above the prop-drilling threshold (default 4).
FE-DATA: Data Fetching	Direct fetch/axios calls appear only in approved API/data-hook modules; every useEffect declares its dependency array	Data-access wrapping ratio = network calls inside approved data-access paths / total detected network calls
FE-DUP: Component/Logic Reusability	No competing feature directories or UI surfaces; no repeated API, form, validation, transformation, state, component, function, or code-block logic	Clone ratio= duplicated-line coverage / total token-bearing production lines
FE-ROUTE: Routing Structure	Route definitions centralised under `src/routes/`; every route resolves to a page component	Average dynamic-parameter count across statically declared routes
FE-STATE: State Location	useState/useReducer must not appear in deep child components; context	Context-provider ratio = context-provider usage count /

	providers appear only at controlled locations	(local state-hook count + context-provider count)
FE-STYLE: Style Isolation	No raw JSX style prop; global styles reside only under 'src/styles/global/'	Style-mixing ratio = files mixing ≥ 2 of {'sx', 'className', 'style', 'styled'} / total files
CROSS-EP: Endpoint Existence	Every frontend API URL resolve to an implemented backend route	Unresolved-endpoint ratio = unresolved frontend call-sites / total detected frontend call-sites
CROSS-TYPE: Request/Type Contract Consistency	Frontend request route params, query fields, and body fields match the backend controller/DTO contract (arity, field existence, required fields, statically resolvable enum/type values)	Weighted contract-field drift density = $\sum(\text{weight}_{(i)} \times \text{mismatch-count}_{(i)})$ across route-param/query/body checks, divided by total contract checkpoints
CROSS-PROP: Change Propagation Completeness	API-facing backend or frontend adapter changes propagate to the resource's existing counterpart surfaces	Change propagation completeness score: successfully propagated changes / total code changes

3.5 Experimental Setup

3.5.1 Controlled Variables

Two independent variables are manipulated. The first is the coding agent, at two levels: Claude Sonnet 4.6, accessed through the Claude Code CLI, and GPT-5.3-codex, accessed through the Codex CLI. Both are current, widely used agentic coding systems, making them representative subjects for this study. The second is prompt structure, also at two levels: a Structured prompt that includes explicit architectural guidance, and a Minimal prompt that includes only the business requirements.

All other factors are held constant across conditions. Each run starts from the same verified starter codebase (3.2) and proceeds through the same fixed task sequence (3.3). The rulepack (3.4) is applied identically to every task snapshot. Both agents run at reasoning effort = high, with subagents permitted, live web retrieval disabled, and account personalisation disabled; the tool environment is locked at the start of the experiment and unchanged throughout.

3.5.2 Prompt Design

Each sprint is delivered to the agent as a markdown document composed of blocks. The Structural and Minimal variants differ in the inclusion of two block; all others are held identical across variants. Table 3.4 summarises the block composition of each variant. For the full prompt descriptions, view the Appendix B.

Table 3.4 Prompt Block Composition

Block	Content	Minimal	Structural
Agent Role	persona and task framing	✓	✓
Codebase Orientation	existing file paths	✓	✓
Problem Statement	feature request and context	✓	✓
Requirements	entity invariants and functional expectations	✓	✓

API Contract	endpoint specifications, DTOs, error codes	X	✓
Architectural Guidance	Rulepack	X	✓
Delivery & Verification Protocol	test execution and completion signal	✓	✓

3.5.3 Execution Protocol

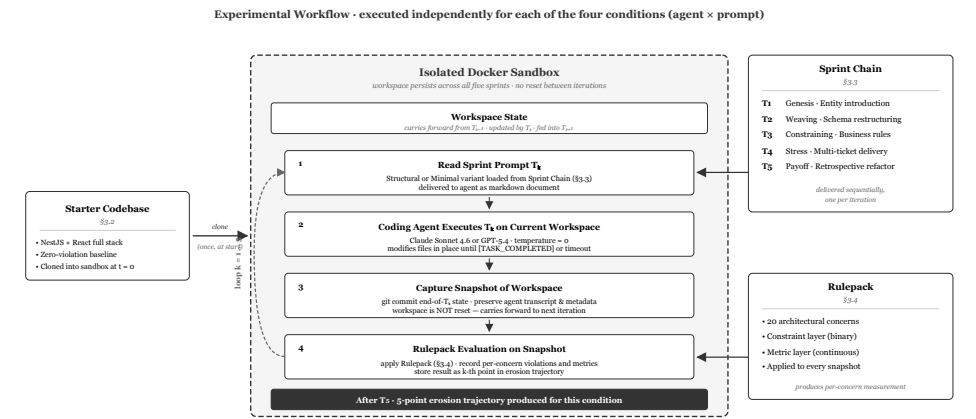


Figure 3.1 Experimental Workflow of the Study

Commented [wl2]: TODO: 更新流程图

The overall workflow is illustrated in Figure 3.1. Each of the four conditions begins from the same verified starter codebase, cloned into a fresh, isolated sandbox. The task chain then drives a three-step loop: at each step k , the agent reads task T_k , executes it on the current workspace, and the resulting workspace is captured as a snapshot and evaluated against the rulepack. The workspace is not reset between steps: the state produced by T_k becomes the exact input to T_{k+1} , so that accumulated deviation can be observed. Each condition is run once, yielding four independent three-point trajectories.

3.6 Threats to Validity

Following the threats-to-validity framework proposed by Wohlin et al. (2012), the discussion below covers threats to construct, internal, external, and conclusion validity.

Construct Validity. First, these nineteen concerns encoded here are a curated set of the primary architectural pressures within the targeted style rather than an exhaustive enumeration; certain minor or design-level deficiencies that resist mechanical detection remain outside the rulepack's scope. Moreover, the specific thresholds embedded in individual rules (e.g., the 300-line component limit, the 5-line duplication threshold) draw on commonly cited practitioner heuristics and remain approximate; alternative thresholds would yield different violation counts.

Internal Validity. Three factors could confound the attribution of observed patterns to the manipulated variables. First, although both the model snapshot and the CLI version are pinned at the start of the experiment, neither fully eliminates provider-side drift. Model pinning fixes the weights but not the surrounding inference stack or server-side moderation layers. CLI pinning freezes the client-side orchestration such as system prompt, tool schemas, and agent loop, but cannot freeze server-executed tools or remotely fetched resources. Any silent revision to these uncontrolled layers could shift agent behaviour in ways this study cannot detect. Second, the

task sequence from greenfield CRUD scaffolding (T1), through relational remodelling (T2), to state-machine enforcement (T3) was fixed to mirror how business systems evolve in practice. Later-task patterns therefore reflect behaviour under one specific ordering; whether a different sequence would produce different accumulated erosion remains untested. Third, the content controls fix only the business-requirement blocks; the surrounding phrasing of the Structured and Minimal variants is one choice among many, and alternative wordings could still bias agent behaviour.

External Validity. The findings apply to the specific class of architecture and technology stack studied: layered client-server applications built on NestJS and React. Generalisation to other architectural styles would require redesigning rulepack concerns to match the constraints those styles impose, since erosion in those architectures' manifests differently. Generalisation to other language ecosystems (Python, Java, Go) would require replacing the JS/TS-specific analysis tooling with equivalents in the target ecosystem. Business-specific rules would similarly need to be added as the rulepack is applied across diversifying domains.

Conclusion Validity. All results in this study derive from single execution runs. Because LLM inference is probabilistic, identical inputs may yield small output variations across runs, introducing measurement noise into the reported violation counts. Our conclusions should therefore be read as indicative rather than exact. Future work could mitigate this threat by averaging results across multiple executions.

Mitigations for these threats, replicating conditions to characterise variance, extending the rulepack to additional architectural styles, and lengthening the task chain beyond three iterations are detailed in 5.4.

3.7 Ethics and Summary

3.7.1 Ethical Considerations

No human participants are involved in this study: all data are produced by software agents interacting with a codebase constructed by the author. Agent invocations were made through the Claude API and the Codex CLI in compliance with Anthropic's and OpenAI's respective Terms of Service. All generated code and captured data are stored locally on the author's-controlled workstation and, upon completion of the study, archived under the University of Warwick's institutional data-management policy. No human participants are involved in this study: all data are produced by software agents interacting with a codebase constructed by the author. All experiment execution reported in this thesis took place after approval was confirmed, consistent with WMG's requirement that data collection occur only once approval is confirmed. A copy of the approval confirmation is provided in Appendix C.

3.7.2 Summary

The methodology presented in this chapter frames the study as a controlled longitudinal experiment. The starter codebase (3.2) provides a verified violation-free baseline; the sprint chain (3.3) prescribes a controlled trajectory of architectural pressure; and the rulepack (3.4) supplies multi-dimensional conformance measurement calibrated for the JavaScript/TypeScript stack. Together, these three components form the benchmark on which the study operates. The experimental setup (3.5) manipulates the coding agent and the prompt structure while holding all other factors constant, executing four conditions once through the five-sprint chain. Threats to validity (3.6) are recognised, mitigated where possible, and otherwise flagged as constraints on interpretation.

Chapter 4: Results and Analysis

4.1 Experimental Runs and Data Quality

The experiment includes four conditions: two coding agents × two prompt strategies. Because the two agent CLIs expose different configuration surfaces, complete parameter parity between them is not achievable. These two configurations were instead constrained to be as similar as practically possible. Both agents were permitted to handle the complexity of full-stack tasks with subagents. Each agent operated inside its sandbox autonomously, without human developer review or intervention. Live web retrieval was disabled for both agents. And to prevent

an agent's account-level personalisation from biasing its responses, all related user configuration and custom agents were disabled as well.

Each condition was executed once across the three-task chain T1–T3, together with one baseline evaluation of the unmodified starter codebase. This yields thirteen harness evaluations in total: twelve agent-run observations plus the baseline. All twelve agent runs completed under the execution protocol defined in §3.5.3: each terminated with exit code 0 and emitted the required [TASK_COMPLETED] marker, with no agent-side errors reported.

The evaluation instrument itself was validated before any experimental run. Every rule was tested against three fixture classes: a compliant implementation, a minimal single-violation implementation, and a near-miss implementation designed to be superficially suspicious while remaining compliant. Metric test coverage was complete (100%) since only rules that passed this fixture validation were admitted into the rulepack used across the experiment evaluations. Table 4.1 records this inventory in full.

Table 4.1 Run inventory

Condition	Task	Execution	Constraints introduced	Constraints Resolves	Constraints cumulative
Claude · Minimal	T1	✓	11	0	11
	T2	✓	9	9	12
	T3	✓	2	5	9
Claude · Structured	T1	✓	9	0	9
	T2	✓	8	8	10
	T3	✓	4	5	9
GPT · Minimal	T1	✓	12	0	12
	T2	✓	31	9	35
	T3	✓	15	16	34
GPT · Structured	T1	✓	11	0	11
	T2	✓	23	9	26
	T3	✓	13	19	20

4.2 Descriptive Overview of Architectural Violations

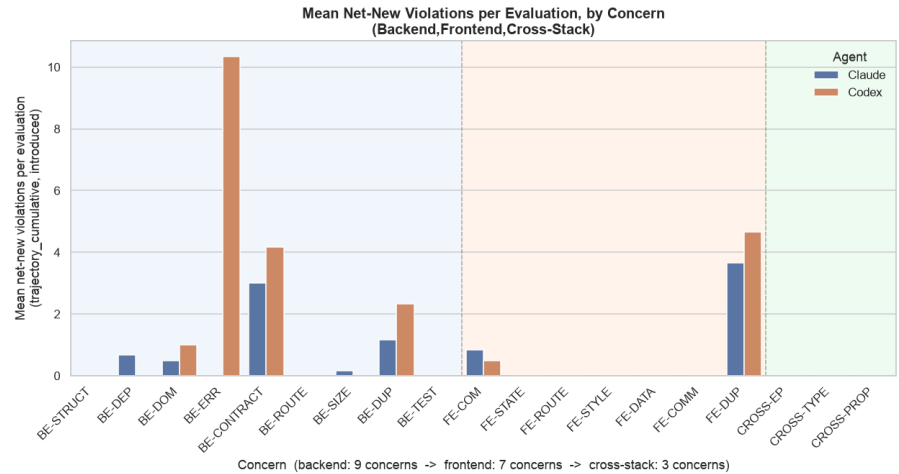


Figure 4.1 Mean net-new violations per run by agent and architectural concern.

Using the formula below, this section computes each agent's mean violation count across all tasks and presents it by concern, as shown in Figure 4.1.

$$\text{mean_net_new_violations}(\text{agent}, \text{concern}) = \frac{\text{total_net_new_violations}(\text{agent}, \text{concern})}{\text{evaluation_count}(\text{agent})}$$

where total net-new violations (agent, concern) sums the net-new constraint findings recorded for that concern across all of the agent's evaluations, and evaluation count (agent) is six, three tasks $\times$ two prompt strategies.

Two patterns hold across both agents. First, violations are far from evenly distributed across the rulepack: eleven of the nineteen concerns: BE-STRUCT, BE-ROUTE, BE-TEST, FE-STATE, FE-ROUTE, FE-STYLE, FE-DATA, FE-COMM, and all three cross-stack concerns (CROSS-EP, CROSS-TYPE, CROSS-PROP) recorded zero net-new violations for both agents across all twelve evaluations. The remaining eight concerns show the entirety of the observed violations, concentrated most heavily in BE-ERR, BE-CONTRACT, and FE-DUP, with BE-DUP, BE-DOM, FE-COM, BE-DEP, and BE-SIZE contributing smaller but non-zero counts. Second, the backend carries substantially more of this signal than the frontend: summed across both agents, backend concerns account for roughly 23 mean violations per evaluation against roughly 10 for frontend concerns, and the cross-stack layer contributes none at all. No cross-stack concern was violated in any of the twelve evaluations.

Within this shared shape, the two agents diverge sharply on individual concerns. The starkest case is BE-ERR: Codex averages 10.33 net-new violations per evaluation, the largest value in the figure, while Claude records almost zero. BE-DEP shows the reverse pattern: Claude averages 0.67, Codex zero. Across the eight concerns with any non-zero signal, Claude records the lower (better) value in five — BE-DOM, BE-ERR, BE-CONTRACT, BE-DUP, and FE-DUP — while Codex records the lower value in the remaining three: BE-DEP, BE-SIZE, and FE-COM. Codex's advantage on FE-COM (0.5 vs Claude's 0.83) is where Codex outperforms Claude, even though Claude's overall violation volume is lower.

4.3 Effect of Prompt Strategy

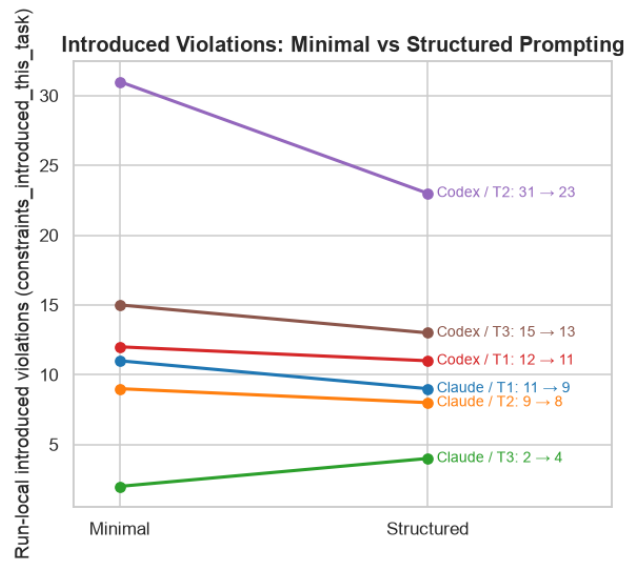


Figure 4.2 Run-Local Introduced Violations: Minimal → Structured.

Using the run-local introduced-violation counts from Table 4.1, Figure 4.2 plots the six Minimal–Structured pairs (three tasks × two agents). Five of the six pairs decline under Structured prompting; the largest decline is Codex at T2 (31 to 23, –8). The one exception is Claude at T3, where violations rise slightly under Structured (2 to 4).

Table 4.2 Minimal vs. Structured: Median Differences

Agent	n	Median Difference (Structured - Minimal)	p (Wilcoxon, two-sided)
Claude	3	-1	1.00
Codex	3	-2	0.25
Claude + Codex	6	-1.5	0.22

Table 4.2 tests whether this decline is statistically reliable. Across all six pairs, the median difference is –1.5, but the two-sided Wilcoxon signed-rank p is 0.22, not significant at the conventional 0.05 threshold. Per agent, Claude's p is 1.00, and Codex's p is 0.25. In short: the direction is consistent, but the sample size is too small to call it a reliable effect.

Table 4.3 Minimal vs. Structured: Per-Rule Net Change by Agent.

Rule ID	Direction	Claude (net)	Codex (net)
BE-ERR-C-002	decreased	0	-10
BE-CONTRACT-C-001	decreased	-4	-1
BE-DOM-C-001	decreased	-2	-3
FE-COM-C-001	decreased	-2	-3
FE-DUP-C-002	decreased	-2	0
BE-SIZE-C-001	decreased	-1	0
BE-DUP-C-003	increased	+7	+4
BE-DUP-C-002	increased	0	+2

Rule ID	Direction	Claude (net)	Codex (net)
BE-DEP-C-004	increased	+2	0
FE-DUP-C-001	increased	+1	0

Table 4.3 summarises the per-pair rule differences underlying Table 4.2 by summing each rule's net change across the three tasks, separately for each agent. Four rules move in the same direction for both agents. Three decreases under Structured for both Claude and Codex: BE-CONTRACT-C-001, BE-DOM-C-001, and FE-COM-C-001. And one increases for both: BE-DUP-C-003 (Claude +7, Codex +4), the rule identified earlier as offsetting much of the aggregate improvement. These four rules represent the part of the Structured effect that is agent independent.

The remaining six rules are agent-specific. Codex's largest single change, BE-ERR-C-002 (−10), belongs to Codex alone. Codex also uniquely contributes BE-DUP-C-002 (+2). Claude's remaining changes are smaller and more distributed: FE-DUP-C-002 (−2), BE-SIZE-C-001 (−1), BE-DEP-C-004 (+2), and FE-DUP-C-001 (+1). The qualitative shape therefore differs: Codex's response to Structured prompting concentrates in one dominant rule, while Claude's is spread thinly across several smaller ones.

4.4 Differences Between Coding Agents

According to Table 4.1, the two agents do not fail by the same amount, and more importantly, they do not fail in the same way. By the end of the task chain, Claude's cumulative net-new violations are identical in both conditions, at 9. Codex's are higher and less consistent: 34 under Minimal, 20 under Structured. Codex introduces two to four times as many violations as Claude, depending on the condition.

As shown in Figure 4.1, eleven of the nineteen concerns show no violation from either agent. Among the remaining eight, two patterns emerge.

Some rules are triggered by both agents, just to different degrees. BE-CONTRACT (Claude 3.0, Codex 4.17), BE-DOM (Claude 0.5, Codex 1.0), BE-DUP (Claude 1.17, Codex 2.33), FE-COM (Claude 0.83, Codex 0.5), and FE-DUP (Claude 3.67, Codex 4.67) all show violations from both agents. Except on FE-COM, where Claude has more violations than Codex, Claude performs better than Codex on every other rule in this group.

Other rules are triggered by only one agent. BE-ERR is triggered by Codex alone (10.33 per evaluation, the largest value in the figure). BE-DEP (0.67) and BE-SIZE (0.17) are triggered by Claude alone; Codex records zero.

4.5 Longitudinal Trajectories Across T1 to T3

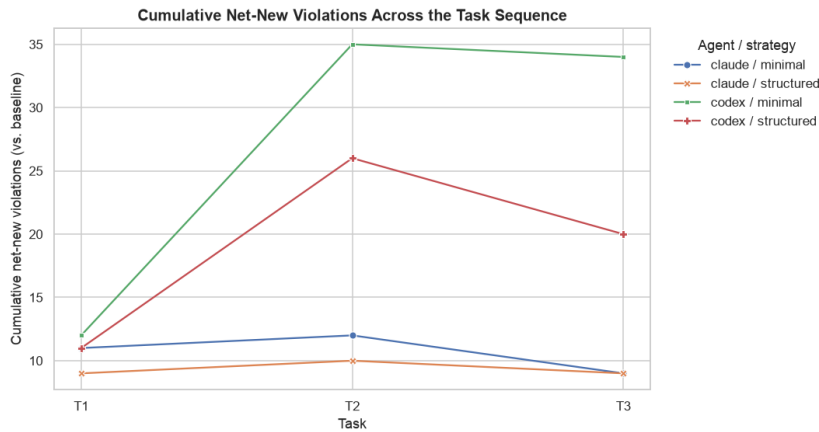


Figure 4.3 Cumulative net-new violation trajectories — T1 to T3 by agent and prompt strategy, with shape classification per line

Figure 4.3 plots cumulative net-new violations at each task checkpoint. At every checkpoint, the harness computes this value by diffing the current codebase state directly against the original, verified-clean baseline, not against the previous task's state.

The size of the T2 spike differs sharply by agent. Claude's increase from T1 to T2 is small in both conditions (11 to 12 under Minimal, 9 to 10 under Structured). Codex's increase is far larger (12 to 35 under Minimal, 11 to 26 under Structured). T2 is the relational-remodelling-and-migration task, and Codex is far more exposed to it than Claude.

All four conditions follow the same shape: violations rise from T1 to T2, then plateau or decline from T2 to T3. None of the four trajectories accelerates — the T2 to T3 slope is not steeper than the T1 to T2 slope. Furthermore, all four conditions improve or hold steady from T2 to T3. Because the baseline itself was verified free of violations (3.2.3), every value in Figure 4.3 represents self-technical debt. T3 shows the agent could self-correct in process, but no trajectory returns to zero. Within this three-task horizon, all four end above the baseline, introducing new violations: 9 (Claude Minimal), 9 (Claude Structured), 34 (Codex Minimal), and 20 (Codex Structured).

4.6 Silent Decay: Constraint and Metric Cross-Classification

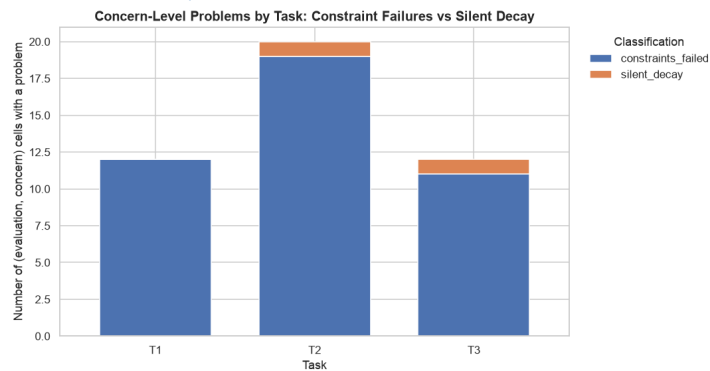


Figure 4.4 Constraint Failures vs. Silent Decay by Task

Each (evaluation, concern) cell is classified into one of four states. A concern is clean if its constraint passes and its metric falls inside the Tukey IQR fence computed across all thirteen evaluations for that metric (low = $Q1 - 1.5 \times IQR$, high = $Q3 + 1.5 \times IQR$). It is silent decay if the constraint passes, but the metric falls outside the fence on the bad side (below the low bound for higher-is-better metrics, above the high bound otherwise). It is a constraint failure if the constraint itself fails, regardless of the metric. The fence is computed independently for each metric, using that metric's own distribution, and no fixed threshold is applied across metrics.

Figure 4.4 aggregates these classifications by task. Constraint failure dominates at every task. Silent decay is a small minority throughout: absent at T1, one cell at T2, one cell at T3. Both silent-decay cells involve the same concern, BE-SIZE, occurring once each for Claude (Minimal, T2) and Codex (Minimal, T3). BE-SIZE shows a three-stage progression for Claude under Minimal prompting: clean at T1, silent decay at T2 (the metric drifts outside its fence while the constraint still passes), and constraint failure at T3 (the same metric's continued drift now crosses the constraint's hard threshold). Codex under Minimal reaches the same second stage, silent decay at T3, just one task later than Claude, but the trajectory ends there. Neither agent shows any deviation on BE-SIZE under Structured prompting. This sequence is consistent with the metric layer surfacing drift before it hardens into a constraint violation.

4.7 File Analyses

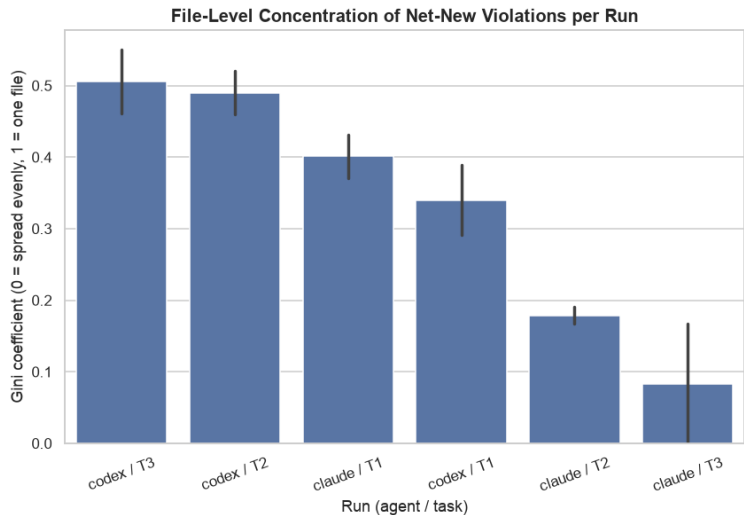


Table 4.5 Violation spatial concentration — files touched, findings, Gini coefficient, and top file with its share, per run. [insert table]

This section explores whether violations cluster in a small number of files or spread across the codebase. For each evaluation, findings are grouped by file, and the Gini coefficient of the resulting per-file counts is computed: 0 means findings are spread evenly across files, 1 means they are concentrated in a single file. This measure is needed because the same total finding count can describe two different situations. For instance, ten findings spread across ten files, or ten findings in one file. And only the second is efficiently fixable by reviewing a single location.

Figure 4.5 shows Gini diverging by agent over the task chain: Claude's concentration falls, Codex's rises. This tracks the BE-ERR monoculture already identified in 4.4: BE-ERR-C-002 can only fire in service-layer files, so Codex's accumulating violations mechanically pile into the same few files, while Claude's violations spread across

rule categories that touch structurally different file types. Table 4.5 names the recurring hotspots directly: `deal.service.ts` alone carries 34.5% of all findings across the study and appears in eleven of twelve evaluations, with `deal.entity.ts` contributing another 15.5% — together, half of everything observed in this chapter traces back to two files.

Concentration of this kind is operationally useful for prioritizing review: a task can touch a dozen files, but if a quarter to four-fifths of its findings sit in just one of them, a reviewer with limited time can prioritise that file first and still catch most of the architectural damage — rather than splitting attention evenly across everything the agent touched.

Table 4.6 Recurring hotspot files(Top 5 across all evaluation)

File	Total findings	Share of all findings	Evaluations present	Evaluations as top file
deal.service.ts	51	34.5%	11 / 12	5
deal.entity.ts	23	15.5%	4 / 12	4
contact.service.ts	16	10.8%	2 / 12	1
contact.entity.ts	9	6.1%	2 / 12	0
dealQueries.js	8	5.4%	4 / 12	0

4.8 Efficiency Outcomes

This section reports duration and cost for T1-T3, the scored trajectory in Table 4.7. Duration is directly reported by both CLIs. Claude's figures are billed amounts reported by the Claude Code CLI, while Codex's are estimated from token counts using published pricing, not a billed figure.

Codex completes T1–T3 in roughly half the time Claude does — about 34–37 minutes in total, against Claude's 68–72 minutes. Codex's estimated cost is also lower, around \$10–11 against Claude's \$16. Structured prompting costs slightly more than Minimal for both agents, about 7–8% more time. This difference is small next to the gap between agents.

On speed and cost, Codex has a clear advantage. On architectural quality, the picture reverses: 4.2–4.7 already showed that Codex introduces two to four times as many violations as Claude. Codex's output appears to trade some architectural quality.

Table 4.7 Duration and cost by agent and prompt strategy — total wall-clock duration and cost accumulated across T1 to T3 in each of the four conditions, with the basis on which the cost figure was obtained.

Agent / strategy	Duration (min)	Cost (USD)	Cost basis
Claude / Minimal	67.65	16.42	Reported
Claude / Structured	72.45	16.44	Reported
Codex / Minimal	33.83	9.65	Estimated
Codex / Structured	36.68	10.96	Estimated

4.9 Summary of Findings

Five findings carry into the discussion. First, architectural erosion under autonomous agents was immediate and mechanism-specific rather than gradual: violations appear at T1 in every condition and recur through a small set of named rules. Second, they concentrate — in three concerns (error handling, duplication, data-contract stability) and in a small number of service files. Third, embedding architectural rules in the prompt produced no statistically defensible reduction in violations, and only a directional improvement for one of the two agents. Fourth, the agents differ more informatively in profile than in volume, with one agent monopolising a rule class

that the other never triggers. Fifth, the silent-decay class was empty because the constraint layer saturated, while the metric layer corroborated and graded severity rather than exposing hidden drift.

Two of these findings are positive results in the strict sense — the concentration of violations and the profile difference between agents — and three are bounded or negative: the prompt-strategy null, the empty silent-decay class, and the absence of verified functional evidence beyond the first task. The chapter reports them at the same level of confidence they were obtained at, and the interpretive weight placed on each in Chapter 5 is calibrated accordingly.

Taken together these results answer the descriptive objectives of the study but leave open what they imply for practice and how far they generalise. Chapter 5 addresses both, together with the limitations that bound them.

Chapter 5: Discussion

5.1 Introduction

This chapter interprets the results reported in Chapter 4 in three movements: what the findings mean for the way architectural erosion under autonomous agents is currently understood, within what limits those meanings hold, and what the limits imply for the next study. The evidence itself is not restated; Chapter 4 owns the figures and statistics, and this chapter argues from them.

5.2 Interpreting the Principal Findings

5.2.1 Architectural erosion under agents is immediate and mechanism-specific

The classical account of architectural erosion describes slow accumulation: drift builds over many maintenance cycles as local decisions accrete into structural decay (Perry and Wolf, 1992; Eick et al., 2001; de Silva and Balasubramaniam, 2012; Li et al., 2022). The results here do not fit that shape. Violations appear at the first task in every condition and recur through a stable set of mechanisms — bypassing the unified exception type, omitting the migration that an entity change requires, and introducing clones — rather than accumulating imperceptibly. Erosion under an autonomous agent looks less like sediment and more like a repeated, identifiable defect class, which is consistent with the emerging agent-focused empirical literature (Zhu, Tsantalis and Rigby, 2026; Agarwal, He and Vasilescu, 2026; Ehsani et al., 2026), and with the industry-scale clone-growth trend reported as grey-literature context by GitClear (2026). The practical consequence is favourable: a failure mode that is immediate and named is easier to gate than one that is gradual and diffuse.

This does not overturn the classical account so much as separate two phenomena that the literature has treated as one. Human-driven erosion is plausibly gradual because the decisions that cause it are distributed across many people and many months, each locally reasonable. Agent-driven erosion, at least in this experiment, is concentrated because the decisions are made by one actor with no memory of the project's conventions beyond what its context window holds, and it makes the same class of decision the same way every time. The consequence for measurement is that the sampling interval matters less than the choice of what to measure: a single evaluation after a single task already reveals the pattern.

5.2.2 Prompt-embedded rules did not secure conformance

The second finding speaks directly to the premise that architectural guidance can be delegated to the prompt. It cannot, at least not on this evidence. The measurable reduction under Structured prompting is not statistically defensible, and even the agent that responded in the expected direction still finished the trajectory with twenty cumulative net-new violations. Stating a rule in the context window makes the rule visible to the agent; it does not make the rule binding on the agent's output. The implication is architectural rather than rhetorical: conformance needs an external enforcement point — a harness or continuous-integration gate of the kind advocated for architecture conformance checking by Bucaioni et al. (2024) — rather than more persuasive instructions. It also reinforces the evaluation critique that motivated this study, namely that pipelines which score only functional outcomes are blind to precisely this class of regression (Jiang et al., 2026).

The distinction worth preserving is between availability and enforcement. The Structured condition made the architecture available to the agent; nothing in the loop made conformance a condition of the agent's own completion criterion. The agent's stopping rule was functional — implement the requirements, report completion — and on that rule every run succeeded. Where a measurement exists but does not participate in the stopping rule, it cannot influence behaviour, which is an argument for placing conformance checks inside the agent's feedback loop rather than after it.

5.2.3 Agents exhibit distinguishable architectural profiles

Read only by volume, the comparison says that one agent is better: Claude records roughly a third of the net-new violations that Codex does. Read by profile, it says something more useful. Codex's findings are dominated by a single rule class — 62 error-handling findings across its six evaluations, almost all from one rule — that Claude

never triggers, while Claude's few findings sit on unit size and dependency direction. The two agents therefore fail in different places, which means review attention should be agent-targeted rather than uniform: an error-handling and migration checklist for one, size and dependency checks for the other. Duplication is the exception, common to both and consistent with clones as a recognised defect predictor (Juergens et al., 2009) and with the aggregate trend in AI-assisted repositories (GitClear, 2026).

Framed for practice, this means agent selection and review policy are not separable decisions. Choosing the lower-volume agent reduces the number of findings but not their kind; choosing the faster agent buys wall-clock time and imports a specific, predictable defect class. Either choice is defensible provided the review effort is aimed where that agent is known to fail, which is a weaker requirement than general architectural review and a stronger one than trusting the agent's completion report.

5.2.4 Degradation follows task structure, with partial self-correction

Lehman's laws predict that unmanaged evolution produces declining quality and increasing complexity over time (Lehman, 1980). Within a three-task horizon that prediction is not borne out: no trajectory accelerates, and the single spike sits at the task designed to apply relational and migration pressure. What the chain reveals instead is that degradation tracks demand — the agent damages the parts of the architecture the task obliges it to touch. The negative run-local deltas at the final task add a second nuance: agents removed more existing violations than they created while implementing new work, so repair capacity exists but is opportunistic rather than systematic. Testing whether Lehman-style feedback appears at all therefore requires longer chains than this design provides.

The self-correction observation deserves one qualification. Agents were not asked to repair anything; the negative deltas at the final task are a by-product of implementing new requirements over code they had themselves written. That makes repair capacity real but unreliable — it happens when the new work passes through the damaged region and not otherwise. It also suggests a cheap intervention consistent with §5.2.2: if incidental exposure produces repair, deliberate exposure by feeding findings back at the start of the next task may produce more.

5.2.5 The empty silent-decay class is a boundary condition, not a refutation

The instrument's distinctive claim is that a two-layer measurement can catch degradation that binary rules miss. No run in this study occupied that class, and the honest interpretation is conditional rather than conclusive. Two explanations remain live. The first is design saturation: tasks purpose-built to stress rulepack concerns, evaluated against a strict rulepack, will trip constraints before metrics can diverge, so the precondition for observing silent decay never obtains. The second is that binary constraints are genuinely sufficient for this class of task and codebase. The present data cannot separate them. What the metric layer did contribute is severity grading and corroboration — outlier metric values coincided with constraint failures in four runs — which is a weaker but real justification for retaining it (Recupito et al., 2024).

5.2.6 Completion claims are not verified outcomes

Finally, the gap between self-reported completion and independent verification deserves emphasis because it is the mechanism by which architectural debt enters a codebase unnoticed. Every run declared its task finished; only one of the four independently checkable runs passed its acceptance suite. Combined with the observation that the faster agent was also the more violation-prone, this reproduces in miniature the volume-versus-quality tension reported for AI-assisted development at scale (Zhu, Tsantalis and Rigby, 2026). The claim is made cautiously: cost and token data are incomplete for one agent, and acceptance suites exist only at the first task.

The broader point is about the epistemics of delegation. An agent's completion report is a statement about its own process, and this experiment provides twelve instances in which that statement was true procedurally and unreliable substantively. Any workflow that treats the report as evidence of correctness inherits that unreliability, and the correction is not better reporting by the agent but independent verification outside it.

5.3 Limitations

The limitations are grouped by validity type, mirroring the framework introduced in §3.6.

Construct validity. The rulepack encodes one architectural interpretation of one business domain. Its nineteen concerns are a defensible selection, not an exhaustive account of architectural integrity, and two of them rely on proxies — the mock ratio standing in for test-construction discipline, and the diff-based change-propagation measure standing in for contract co-evolution. The cross-stack rulepack remains experimental and recorded zero findings throughout, so the study cannot distinguish between agents that preserved front-to-back contracts and an instrument that is insensitive to their violation.

Internal validity. The Structured prompt was not ablated, so the effect of its architectural contract cannot be separated from the effect of its rule vocabulary. The baseline carries five pre-existing frontend findings; net-delta accounting removes them from the counts but cannot rule out that agents imitated the inherited patterns they observed in the starter codebase. The two command-line interfaces are also not parameter-identical: reasoning effort was set to high for both, but the surrounding harness surfaces differ in ways the study could constrain only approximately.

External validity. The experiment covers one codebase, one architectural style — a layered TypeScript and JavaScript monolithic customer-relationship-management application — and two agents at fixed model snapshots. No claim is made about microservice, event-driven or modular-monolith architectures, or about later model releases.

Statistical conclusion validity. Each condition was executed once, without replication. The Wilcoxon floor of $p = 0.25$ at three pairs means no per-agent comparison could have reached significance. Trajectories rest on three points, acceptance suites exist only at the first task, and the tasks differ in size (approximately 24, 40 and 39 requirements), which confounds cross-task comparison of absolute counts. The Judgments layer is designed but not operationalised, so no dimension of quality requiring interpretation was measured at all.

Taken together, these limitations constrain the study's claims in a specific direction. They do not undermine the observation that erosion occurred, which is a direct reading of rule-level findings against a documented architecture. They do constrain the generality of the profile differences between agents, the interpretation of the prompt-strategy null, and any statement about magnitude. The results are best read as a characterisation of what these two agents did to this architecture under this chain, offered with an instrument and a design that another study can re-run.

5.4 Future Work

The extensions below are ordered by dependency, each addressing a limitation named above. First, replication: multiple sessions per condition would convert the directional prompt-strategy result into an inferential one and is the precondition for everything that follows. Second, implementing the Judgments layer, which would permit the architecture-by-correctness quadrant analysis the instrument was designed to support and close the gap between conformance and quality. Third, ablation of the Structured prompt's individual blocks, to identify which guidance actually changes agent behaviour. Fourth, longer task chains, including composite tasks beyond T3, to test whether Lehman-style acceleration appears once the horizon exceeds three steps. Fifth, enforcement in the loop: feeding harness findings back to the agent mid-trajectory, which is the design implied by the failure of prompt-only guidance and the most direct route from measurement to mitigation. Sixth, additional rulepacks, architectural styles and agents, to establish how far the profile differences observed here generalise. Seventh, revisiting the four cross-stack categories pruned during instrument development, together with the agent self-review probe that was designed and piloted but not retained in the final dataset, both of which would sharpen the instrument's coverage of contract-level erosion.

Two of these extensions are cheap and would materially change what can be claimed. Replication requires no new engineering, only repeated execution of an existing pipeline, and would move the prompt-strategy

comparison from directional to inferential. Enforcement in the loop requires only that findings already produced by the harness be returned to the agent between tasks, and it tests the study's central practical hypothesis directly: that conformance responds to feedback where it does not respond to instruction. The remainder — the Judgments layer, additional architectures, longer chains — are larger programmes of work that depend on those two being done first.

5.5 Summary

Architectural erosion under autonomous agents is real, but it is structured rather than diffuse: immediate, concentrated in a few rule classes and files, and driven by what each task obliges the agent to change. Prompt-embedded architectural guidance did not secure conformance, and self-reported completion did not predict verified functional outcomes. The actionable consequence is a pairing: external, automated conformance enforcement rather than instruction alone, and agent-profiled review that looks first where the particular agent is known to fail.

Chapter 6: Conclusion

6.1 Summary of the Study

This study addressed a gap between how autonomous coding agents are evaluated and how software actually decays. Benchmarks for agentic code generation score functional outcomes — tests passed, issues resolved — while architectural conformance, the property that determines whether a codebase remains maintainable after repeated modification, goes unmeasured. The study asked whether autonomous agents erode architectural integrity when they modify an existing system across successive tasks, and whether making architectural rules explicit in the prompt prevents that erosion.

The apparatus built to answer that question has three components. A starter codebase provides a verified reference system with a documented architecture. An entangled task chain applies controlled architectural pressure across three sequential tasks executed against a workspace that is never reset, so each task inherits the previous task's output. A rulepack organises nineteen architectural concerns across a binary constraint layer, a continuous metric layer and a designed-but-unimplemented judgment layer, mapping each concern to a named rule. Four conditions — two agents crossed with two prompt strategies — were executed through the chain, producing thirteen harness evaluations analysed under a five-stage protocol.

The design choice that distinguishes the study from existing agent benchmarks is the refusal to reset the workspace between tasks. Benchmarks that evaluate each task from a clean checkout measure capability; by carrying each task's output forward as the next task's input, this study measures what happens to a codebase that is modified repeatedly, which is the condition under which architectural erosion is defined. The cost of that choice is a small number of long trajectories rather than a large number of independent trials, and the analysis is framed to respect it.

6.2 Answers to the Research Question and Objectives

On the central question, the answer is affirmative and specific. Autonomous agents did erode architectural integrity while completing functionally scoped tasks, and making the architecture explicit in the prompt did not prevent it. Erosion appeared at the first task in every condition, concentrated in a small number of rule classes and files, and tracked the architectural pressure that each task applied.

The first two objectives were delivered as artefacts. The measurement framework separates mechanically decidable constraints from continuous drift signals and from properties requiring interpretation, and the instrument implements the first two layers against a JavaScript and TypeScript full-stack codebase. Their reliability is evidenced by the execution record: zero metric errors and zero scope errors across thirteen evaluations, complete metric coverage for all agent runs, and a baseline self-check delta of zero.

The third objective — characterising how architectural integrity evolves across sequential agent-executed tasks — is answered descriptively but precisely. Erosion is immediate rather than gradual, concentrated rather than diffuse, task-driven rather than monotonic, and partially self-correcting: at the final task three of four trajectories resolved more existing violations than they introduced.

The fourth objective concerned prompt strategy. Structured prompting produced no statistically defensible reduction in net-new violations. The direction of the effect was consistent for one agent and absent for the other, and at three pairs per agent the significance test could not have returned a conventional threshold regardless of effect size. The finding is therefore reported as directional, with the negative conclusion stated plainly: architectural instruction in the prompt was not sufficient.

The fifth objective sought practitioner guidance, which the results support in three parts. Conformance should be enforced externally, by a harness or continuous-integration gate, rather than delegated to prompt wording. Review should be agent-profiled, because the two agents failed in different rule classes rather than at different

rates. And the checks with the highest yield in this experiment were narrow and specific: unified exception handling, migrations accompanying entity changes, and duplication.

Read against the research question as originally posed, the answer therefore has two halves that must be kept together. Agents do erode architecture, and the erosion is measurable, localisable and attributable to named rules — which is the optimistic half, because it makes the problem tractable. Explicit architectural instruction did not prevent it, and the agents' own completion reports did not reveal it — which is the cautionary half, because it rules out the two cheapest mitigations a practitioner would reach for first.

6.3 Contributions

The study makes three contributions. The first is a reusable, rule-linked conformance harness for full-stack JavaScript and TypeScript systems that combines binary constraints with continuous metrics and reports findings traceable to named rules. The second is an entangled-task longitudinal design that isolates the effect of prompt-level architectural guidance while holding the codebase trajectory fixed, and which locates erosion at the task that was designed to provoke it. The third is a controlled empirical characterisation of architectural erosion profiles for two contemporary coding agents under that design, including the profile contrast that volume comparisons obscure. Each contribution is bounded by the limitations set out in §5.3, in particular the single codebase, the single architectural style and the absence of replication.

A fourth outcome is methodological rather than empirical and is offered as a secondary contribution. The study demonstrates that architectural conformance can be measured on agent output at the granularity of a single task, cheaply enough to run after every task, and reported in a form a reviewer can act on: a named rule, a file, and a count. This matters because the alternative currently available to practitioners is periodic manual architecture review, which is too slow to keep pace with an agent that can complete a multi-file change in fifteen minutes. Whether the specific rulepack used here transfers to other stacks is an open question; that the measurement is tractable at this cadence is not.

6.4 Closing Remarks

The question raised in §1.3 was how much of software maintenance can be delegated to an autonomous agent. On this evidence the answer is not a quantity but a condition. Agents completed every task they were given and reported success in every case, while accumulating structural debt that no functional check would have surfaced and that instruction alone did not prevent. Delegation is therefore viable to the extent that conformance is measured and enforced outside the agent's own account of its work. Measurement, not trust, is the precondition for deployment.

That conclusion is deliberately modest in scope and specific in content. It does not say that agents should not be used for maintenance work; every task in this experiment was completed, and one agent completed them in roughly half the wall-clock time of the other. It says that the artefact an agent returns should be judged on two axes rather than one, and that the second axis — conformance to the architecture the codebase already commits to — is measurable today with rules that a project can write down and a harness can check on every change. The instrument built for this study is one demonstration that the measurement is tractable; the results are a demonstration that it is necessary.

References

- Agarwal, S., He, H. and Vasilescu, B. (2026) 'AI IDEs or autonomous agents? Measuring the impact of coding agents on software development'. arXiv. Available at: <https://arxiv.org/abs/2601.13597>.
- Bucaioni, A. *et al.* (2024) 'Continuous Conformance of Software Architectures', *2024 IEEE 21st International Conference on Software Architecture (ICSA)*. *2024 IEEE 21st International Conference on Software Architecture (ICSA)*, pp. 112–122. Available at: <https://doi.org/10.1109/ICSA59870.2024.00019>.
- Deng, X. *et al.* (2025) 'SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks?' arXiv. Available at: <https://doi.org/10.48550/arXiv.2509.16941>.
- Eick, S.G. *et al.* (2001) 'Does code decay? Assessing the evidence from change management data', *IEEE Transactions on Software Engineering*, 27(1), pp. 1–12. Available at: <https://doi.org/10.1109/32.895984>.
- Esposito, M. *et al.* (2026) 'Generative AI for software architecture. Applications, challenges, and future directions', *Journal of Systems and Software*, 231, p. 112607. Available at: <https://doi.org/10.1016/j.jss.2025.112607>.
- Jalil, S. (2025) 'The Transformative Influence of LLMs on Software Development & Developer Productivity', *2025 International Conference on Artificial Intelligence, Computer, Data Sciences and Applications (ACDSA)*. *2025 International Conference on Artificial Intelligence, Computer, Data Sciences and Applications (ACDSA)*, pp. 1–10. Available at: <https://doi.org/10.1109/ACDSA65407.2025.11166110>.
- Jiang, J. *et al.* (2026) 'A Survey on Large Language Models for Code Generation', *ACM Trans. Softw. Eng. Methodol.*, 35(2), p. 58:1-58:72. Available at: <https://doi.org/10.1145/3747588>.
- Le, T. *et al.* (2026) 'SWE-EVO: Benchmarking Coding Agents in Long-Horizon Software Evolution Scenarios'. arXiv. Available at: <https://doi.org/10.48550/arXiv.2512.18470>.
- Li, H., Zhang, H. and Hassan, A.E. (2025) 'The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering'. arXiv. Available at: <https://doi.org/10.48550/arXiv.2507.15003>.
- Li, S. *et al.* (2024) 'Assessing the Performance of AI-Generated Code: A Case Study on GitHub Copilot', *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*. *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*, pp. 216–227. Available at: <https://doi.org/10.1109/ISSRE62328.2024.00030>.
- Li, Z. *et al.* (2014) 'An empirical investigation of modularity metrics for indicating architectural technical debt', *Proceedings of the 10th international ACM Sigsoft conference on Quality of software architectures*. New York, NY, USA: Association for Computing Machinery (QoSA '14), pp. 119–128. Available at: <https://doi.org/10.1145/2602576.2602581>.
- Liao, D. *et al.* (2024) 'A³-CodGen: A Repository-Level Code Generation Framework for Code Reuse With Local-Aware, Global-Aware, and Third-Party-Library-Aware', *IEEE Transactions on Software Engineering*, 50(12), pp. 3369–3384. Available at: <https://doi.org/10.1109/TSE.2024.3486195>.
- Mo, R. *et al.* (2025) 'Assessing and Analyzing the Correctness of GitHub Copilot's Code Suggestions', *ACM Trans. Softw. Eng. Methodol.*, 34(7), p. 194:1-194:32. Available at: <https://doi.org/10.1145/3715108>.
- Perry, D.E. and Wolf, A.L. (1992) 'Foundations for the study of software architecture', *ACM SIGSOFT Software Engineering Notes*, 17(4), pp. 40–52. Available at: <https://doi.org/10.1145/141874.141884>.

Recupito, G. *et al.* (2024) 'Technical debt in AI-enabled systems: On the prevalence, severity, impact, and management strategies for code and architecture', *Journal of Systems and Software*, 216, p. 112151. Available at: <https://doi.org/10.1016/j.jss.2024.112151>.

Siddiq, M.L. *et al.* (2024) 'SALLM: Security Assessment of Generated Code', *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering Workshops. ASEW '24: 39th IEEE/ACM International Conference on Automated Software Engineering Workshops*, Sacramento CA USA: ACM, pp. 54–65. Available at: <https://doi.org/10.1145/3691621.3694934>.

de Silva, L. and Balasubramaniam, D. (2012) 'Controlling software architecture erosion: A survey', *Journal of Systems and Software*, 85(1), pp. 132–151. Available at: <https://doi.org/10.1016/j.jss.2011.07.036>.

Wan, Z. *et al.* (2023) 'Software Architecture in Practice: Challenges and Opportunities', *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. New York, NY, USA: Association for Computing Machinery (ESEC/FSE 2023), pp. 1457–1469. Available at: <https://doi.org/10.1145/3611643.3616367>.

Wang, C. *et al.* (2025) 'Teaching Code LLMs to Use Autocompletion Tools in Repository-Level Code Generation', *ACM Trans. Softw. Eng. Methodol.*, 34(7), p. 191:1–191:27. Available at: <https://doi.org/10.1145/3714462>.

Xia, C.S. *et al.* (2024) 'Agentless: Demystifying LLM-based Software Engineering Agents'. arXiv. Available at: <https://doi.org/10.48550/arXiv.2407.01489>.

Zeng, Z. *et al.* (2025) 'Benchmarking and Studying the LLM-based Agent System in End-to-End Software Development'. arXiv. Available at: <https://doi.org/10.48550/arXiv.2511.04064>.

Appendices

Appendix A: Rulepack Rules

Appendix B: Prompt Templates

Appendix C: Ethical Approval

Appendix D: Experiment Config